

## ■ Lab 12 — Modules

---

### ■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Créé des **modules** réutilisables pour EC2 et Security Group
- Compris le **flux d'information** entre module parent et modules enfants
- Appelé un module depuis le code principal

### ■ Étape 1 — Structure des fichiers

---

```
cd labs/lab12-modules
```

```
lab12-modules/  
■■■ backend.tf  
■■■ versions.tf  
■■■ provider.tf  
■■■ variables.tf  
■■■ locals.tf  
■■■ main.tf  
■■■ outputs.tf  
■■■ modules/  
    ■■■ ec2/  
        ■■■ main.tf  
        ■■■ variables.tf  
        ■■■ outputs.tf  
    ■■■ security_group/  
        ■■■ main.tf  
        ■■■ variables.tf  
        ■■■ outputs.tf
```

### ■ Étape 2 — Module Security Group

---

`modules/security_group/variables.tf`

```
variable "name" {
  description = "Nom du Security Group"
  type       = string
}

variable "description" {
  description = "Description du Security Group"
  type       = string
  default    = "Security Group géré par Terraform"
}

variable "ingress_rules" {
  description = "Liste des règles ingress"
  type = list(object({
    port      = number
    protocol   = string
    description = string
  }))
  default = []
}

variable "tags" {
  description = "Tags à appliquer"
  type       = map(string)
  default    = {}
}
```

`modules/security_group/main.tf`

```
resource "aws_security_group" "this" {
  name          = var.name
  description    = var.description

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port    = ingress.value.port
      to_port      = ingress.value.port
      protocol     = ingress.value.protocol
      cidr_blocks = ["0.0.0.0/0"]
      description = ingress.value.description
    }
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = var.tags
}
```

## ``modules/security_group/outputs.tf``

```
output "security_group_id" {
  description = "ID du Security Group"
  value       = aws_security_group.this.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value       = aws_security_group.this.name
}
```

## ■ Étape 3 — Module EC2

### ``modules/ec2/variables.tf``

```
variable "name" {
  description = "Nom de l'instance"
  type        = string
}

variable "ami_id" {
  description = "ID de l'AMI"
  type        = string
}

variable "instance_type" {
  description = "Type d'instance"
  type        = string
  default     = "t2.micro"
}

variable "security_group_id" {
  description = "ID du Security Group"
  type        = string
}

variable "tags" {
  description = "Tags à appliquer"
  type        = map(string)
  default     = {}
}
```

### `modules/ec2/main.tf`

```
resource "aws_instance" "this" {
  ami                = var.ami_id
  instance_type      = var.instance_type
  vpc_security_group_ids = [var.security_group_id]

  tags = merge(var.tags, {
    Name = var.name
  })
}
```

### `modules/ec2/outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance"
  value       = aws_instance.this.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value       = aws_instance.this.public_ip
}
```

## ■ Étape 4 — Fichiers racine

### ``backend.tf``

```
terraform {  
  backend "s3" {  
    bucket      = "tf-training-<votre-prenom>-982908300187"  
    key         = "terraform.tfstate"  
    region     = "eu-west-1"  
    use_lockfile = true  
    encrypt     = true  
  }  
}
```

■ ■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

### ``versions.tf``

```
terraform {  
  required_version = ">= 1.15.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

### ``provider.tf``

```
provider "aws" {  
  region = var.region  
}
```

### ``variables.tf``

```
variable "username" {
  description = "Votre prenom"
  type        = string
}

variable "region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-1"
}
```

### `locals.tf`

```
locals {
  prefix = "lab12-${var.username}"

  common_tags = {
    Lab        = "lab12"
    Username    = var.username
    ManagedBy  = "Terraform"
  }
}
```

### `main.tf`

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

module "security_group" {
  source = "../modules/security_group"

  name          = "${local.prefix}-sg"
  description    = "Security Group Lab 12 - ${var.username}"
  tags           = local.common_tags

  ingress_rules = [
    { port = 22, protocol = "tcp", description = "SSH" },
    { port = 80, protocol = "tcp", description = "HTTP" },
    { port = 443, protocol = "tcp", description = "HTTPS" },
  ]
}

module "ec2" {
  source = "../modules/ec2"

  name          = "${local.prefix}-instance"
  ami_id        = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
  security_group_id = module.security_group.security_group_id

  tags           = local.common_tags
}
```

## outputs.tf

```
output "instance_id" {
  description = "ID de l'instance"
  value       = module.ec2.instance_id
}

output "instance_public_ip" {
  description = "IP publique"
  value       = module.ec2.instance_public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value       = module.security_group.security_group_id
}
```

## ■ Étape 5 — Déployer

```
# terraform init télécharge aussi les modules locaux
terraform init
terraform apply -var="username=<votre-prenom>"
```

Vérifiez les outputs :

```
terraform output
terraform state list
```

■ Les ressources dans le state apparaissent avec le préfixe `module.ec2.` et `module.security_group.`

## ■ Étape 6 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

## ■ Validation du Lab

| # | Critère                                                                                                     | Validé |
|---|-------------------------------------------------------------------------------------------------------------|--------|
| 1 | Modules <code>ec2</code> et <code>security_group</code> créés dans <code>modules/</code>                    | ■      |
| 2 | <code>terraform init</code> télécharge les modules locaux                                                   | ■      |
| 3 | <code>terraform apply</code> crée les ressources via les modules                                            | ■      |
| 4 | <code>terraform state list</code> affiche <code>module.ec2.*</code> et <code>module.security_group.*</code> | ■      |
| 5 | Output <code>instance_id</code> utilise <code>module.ec2.instance_id</code>                                 | ■      |
| 6 | <code>terraform destroy</code> supprime toutes les ressources                                               | ■      |



■ **\*\*Fin du Lab 12 — Votre code est modulaire et réutilisable !\*\***

*Le Lab 13 introduit les **\*\*workspaces\*\*** pour gérer plusieurs environnements.*